

Computer Vision

Department of SE
University of the Punjab

Recitation Test 1

Total marks: 20 marks

- **First** rename this file to include your roll number and your name. For example, SE_Ass2_BCSF22M599_Asim.
- Fill in your code.
- Execute all cells to generate outputs.
- For grading, I will look at your code only when the output is incorrect. If the output is correct, I will assume your code is correct as well.
- Once you are satisfied that your outputs will get good marks, download the notebook in .pdf format.
- Submit both the .ipynb and .pdf files in the Google Classroom.

Python Basics

Take weight in kgs and convert it into pounds. 1 kg = 2.20462 pounds. **(0.5 mark)**

```
kg=float(input("Enter the weight in Kg"))  
pounds=kg*2.20462  
print('Weight in pounds:',pounds)
```

```
Enter the weight in Kg1  
Weight in pounds: 2.20462
```

Calculate the cost of all the items in a shopping cart. **(0.5 mark)**

```
prices=[20,50,80,10,56,89]  
#using fun sum  
cost=sum(prices)  
print("cost of items: ",cost)
```

```
cost of items: 305
```

Write a function that returns the maximum of two numbers. **(0.5 mark)**

```
def maximum(a,b):  
    if(a>b):  
        return a  
    else:  
        return b  
  
print(maximum(10,25))  
  
#built in function  
#maximumValue=max(2,5)  
#print(maximumValue)
```

```
25  
5
```

Write a function called **deepmind** that takes a number **(2 marks)**

- If the number is divisible by 3, it should return deep.
- If it is divisible by 5, it should return mind.
- If it is divisible by both 3 and 5, it should return deepmind.
- Otherwise, it should return the same number.

```
def deepmind(num):
    if(num%3==0 and num%5==0):
        return "deepmind"
    elif(num%3==0):
        return "deep"
    elif(num%5==0):
        return "mind"
    else:
        return num
answer=deepmind(23)
print(answer)
```

23

listA = [1,2,3,4,5,6,7,8,9,10]

If any element of **listA** is smaller than 5, replace it with 0. And if any element of **list A** is bigger than 5, replace it with 1. **(1 mark)**

```
listA = [1,2,3,4,5,6,7,8,9,10]
for i in range(len(listA)):
    if(listA[i]>5):
        listA[i]=1
    elif(listA[i]<5):
        listA[i]=0
print(listA)
```

[0, 0, 0, 0, 5, 1, 1, 1, 1, 1]

Compute the square of **listA** elements in one line. **(1 mark)**

```
listA = [1,2,3,4,5,6,7,8,9,10]
squared = [x**2 for x in listA]

print(squared)
```

[1, 4, 9, 16, 25, 36, 49, 64, 81, 100]

Concatenate b1 and b2. **(0.5 mark)**

```
b1 = ['Hello', 'in','first']
b2 = ['Students','the','recitation']
concatenated_b = b1 + b2

print(concatenated_b)
```

['Hello', 'in', 'first', 'Students', 'the', 'recitation']

Create a dictionary of student **Ali** where the keys are courses and values are the total and obtained marks in each course. **(1 mark)**

```
Ali={
    "Math": {"Total": 100, "Obtained": 85},
    "Physics": {"Total": 100, "Obtained": 78},
    "English": {"Total": 100, "Obtained": 90}
}
print(Ali)
```

{'Math': {'Total': 100, 'Obtained': 85}, 'Physics': {'Total': 100, 'Obtained': 78}, 'English': {'Total': 100, 'Obtained': 90}}

Create a class 'calculator' with the following functions to compute i) addition, ii) subtraction, iii)multiplication, iv)division and v)square between two numbers. **(1 mark)**

```
class calculator:

    def addition(self, a, b):
        return a + b

    def subtraction(self, a, b):
        return a - b

    def multiplication(self, a, b):
        return a * b

    def division(self, a, b):
        if b != 0:
            return a / b
        else:
            return "Cannot divide by zero"

    def square(self, a):
        return a ** 2

# Example usage
calc = calculator()

print(calc.addition(10, 5))
print(calc.subtraction(10, 5))
print(calc.multiplication(10, 5))
print(calc.division(10, 5))
print(calc.square(4))
```

```
15
5
50
2.0
16
```

▼ Numpy Basics

Make two 1D arrays 'u' and 'v' consisting of 3 uniform random numbers between 0 and 2 each. **(0.5 mark)**

```
import numpy as np
u=np.random.uniform(0,2,3)
v=np.random.uniform(0,2,3)
print("u:",u)
print("v:",v)
```

```
u: [1.67172449 0.89406292 0.04120826]
v: [0.39093388 0.92462833 1.64186067]
```

Compute the lengths of vectors 'u' and 'v'. Vector lengths are also called *magnitudes* or *norms*. **(0.5 mark)**

```
u_len=np.linalg.norm(u)
v_len=np.linalg.norm(v)

print("Length of u:", u_len)
print("Length of v:", v_len)
```

```
Length of u: 1.8962355900357069
Length of v: 1.9244410319730454
```

Compute the dot-product of 'u' and 'v'. **(0.5 mark)**

```
dot_uv=np.dot(u,v)
print("Dot product:", dot_uv)
```

```
Dot product: 1.5478678600112672
```

Make a 2D array 'M' of size (2,3) containing uniform random numbers between 0 and 1. **(0.5 mark)**

```
M=np.random.uniform(0,1,(2,3))
print(M)
```

```
[[0.75165105 0.94404022 0.57702431]
 [0.51006113 0.9712309 0.55760254]]
```

Compute the matrix-vector product of 'M' and 'u'. (0.5 mark)

```
result = np.dot(M, u)
print("Matrix-vector product:", result)
```

```
Matrix-vector product: [2.12436299 1.74400105]
```

Make an array 'A' consisting of a (3,4) array, a (2,4) array and a (4,4) array. All 3 arrays should contain random integers less than 9. (1 mark)

```
A1=np.random.randint(0,9,(3,4))
A2=np.random.randint(0,9,(2,4))
A3 = np.random.randint(0, 9, (4,4))
```

```
A = [A1, A2, A3]
```

```
for arr in A:
    print(arr, '\n')
```

```
[[4 3 5 7]
 [6 3 8 5]
 [7 0 0 8]]
```

```
[[5 8 2 3]
 [8 3 3 4]]
```

```
[[2 8 6 6]
 [2 1 1 6]
 [8 3 6 0]
 [6 0 7 3]]
```

Pad the three arrays in 'A' to make a tensor array 'B'. Padding should be at the top with constant value of -1. (0.5 marks)

```
B=[np.pad(arr, ((4-arr.shape[0],0),(0,0)),constant_values=-1) for arr in A]
for arr in B:
    print(arr, "\n")
```

```
[[ -1 -1 -1 -1]
 [ 4  3  5  7]
 [ 6  3  8  5]
 [ 7  0  0  8]]
```

```
[[ -1 -1 -1 -1]
 [ -1 -1 -1 -1]
 [ 5  8  2  3]
 [ 8  3  3  4]]
```

```
[[2 8 6 6]
 [2 1 1 6]
 [8 3 6 0]
 [6 0 7 3]]
```

Stack the arrays in 'A' vertically. (0.5 mark)

```
A_vertical=np.vstack(A)
print(A_vertical)
```

```
[[4 3 5 7]
 [6 3 8 5]
 [7 0 0 8]
 [5 8 2 3]
 [8 3 3 4]
 [2 8 6 6]
 [2 1 1 6]
 [8 3 6 0]
 [6 0 7 3]]
```

Stack the arrays in 'B' horizontally using only 1 line of code containing a for loop. (0.5 marks)

```
B_horizontal = np.hstack([arr for arr in B])
print(B_horizontal)
```

```
[[ -1 -1 -1 -1 -1 -1 -1 -1  2  8  6  6]
 [  4  3  5  7 -1 -1 -1 -1  2  1  1  6]
 [  6  3  8  5  5  8  2  3  8  3  6  0]
 [  7  0  0  8  8  3  3  4  6  0  7  3]]
```

Create a matrix **A** of size (3,5) with 0 mean and 5 standard deviation. **(0.5 mark)**

```
A = np.random.normal(0, 5, (3,5))
print(A)
```

```
[[-6.68286752 -4.12116486  6.57235758  2.90068712 -7.78003316]
 [10.72169188 -2.49412099  1.13132238  6.07191475  6.51968856]
 [ 2.69225221 -0.55553463  2.44917288 -4.84966968  5.98597548]]
```

- Set all the elements in first row of **A** to 7.
- Set all the elements in second row of **A** to -2.
- Set the element in third row and second column of **A** to 9. **(1 mark)**

```
A[0, :] = 7
A[1, :] = -2
A[2, 1] = 9
```

```
print(A)
```

```
[[ 7.         7.         7.         7.         7.        ]
 [-2.        -2.        -2.        -2.        -2.        ]
 [ 2.69225221  9.         2.44917288 -4.84966968  5.98597548]]
```

For each row of matrix **A**, subtract the mean of that row. **(1 mark)**

```
A = A - A.mean(axis=1, keepdims=True)
print(A)
```

```
[[ 0.         0.         0.         0.         0.        ]
 [ 0.         0.         0.         0.         0.        ]
 [-0.36329397  5.94445382 -0.6063733  -7.90521585  2.9304293  ]]
```

Create a vector **a** of size 30 with values ranging from 60 to 89. **(0.5 mark)**

```
a = np.arange(60, 90)
print(a)
```

```
[60 61 62 63 64 65 66 67 68 69 70 71 72 73 74 75 76 77 78 79 80 81 82 83
 84 85 86 87 88 89]
```

Create a (5,5) matrix **M** with values ranging from 0 to 24. **(0.5 mark)**

```
M = np.arange(25).reshape(5,5)
print(M)
```

```
[[ 0  1  2  3  4]
 [ 5  6  7  8  9]
 [10 11 12 13 14]
 [15 16 17 18 19]
 [20 21 22 23 24]]
```

Make a (3,3) matrix **N** with 5 on the border and 0 inside. **(0.5 mark)**

```
N = np.ones((3,3)) * 5
N[1,1] = 0
print(N)
```

```
[[5. 5. 5.]
 [5. 0. 5.]
 [5. 5. 5.]]
```

Create a matrix **P** of size (3,4) with random integer values. **(0.5 mark)**

```
P = np.random.randint(0, 10, (3,4))
print(P)
```

```
[[2 1 2 2]
 [8 2 5 7]
 [5 5 9 1]]
```

Create a 3-D array **L** with five on the diagonal and zeros elsewhere. **(0.5 mark)**

```
L = np.zeros((3,3,3))
np.fill_diagonal(L, 5)
print(L)

[[[5. 0. 0.]
  [0. 0. 0.]
  [0. 0. 0.]]

 [[0. 0. 0.]
  [0. 5. 0.]
  [0. 0. 0.]]

 [[0. 0. 0.]
  [0. 0. 0.]
  [0. 0. 5.]]]
```

Create a class 'calculator' with the following functions to compute: **(3 marks)**

- the dot product between two vectors
- the cross product between two vectors
- the length (norm) of the vector
- matrix-vector product of a matrix and vector
- the mean squared error between two vectors

Show output that confirms that you have implemented all functions correctly.

```
class calculator:

    def dot_product(self, a, b):
        return np.dot(a, b)

    def cross_product(self, a, b):
        return np.cross(a, b)

    def norm(self, a):
        return np.linalg.norm(a)

    def mat_vec(self, M, v):
        return np.dot(M, v)

    def mse(self, a, b):
        return np.mean((a-b)**2)

calc = calculator()

x = np.array([1,2,3])
y = np.array([4,5,6])
M = np.array([[1,2,3],[4,5,6]])

print("Dot:", calc.dot_product(x,y))
print("Cross:", calc.cross_product(x,y))
print("Norm:", calc.norm(x))
print("Matrix-Vector:", calc.mat_vec(M,x))
print("MSE:", calc.mse(x,y))
```

```
Dot: 32
Cross: [-3  6 -3]
Norm: 3.7416573867739413
Matrix-Vector: [14 32]
MSE: 9.0
```